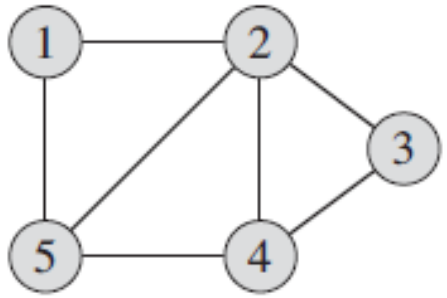
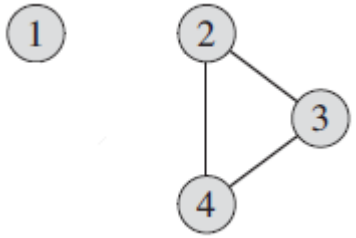
Graphs

What is a Graph?

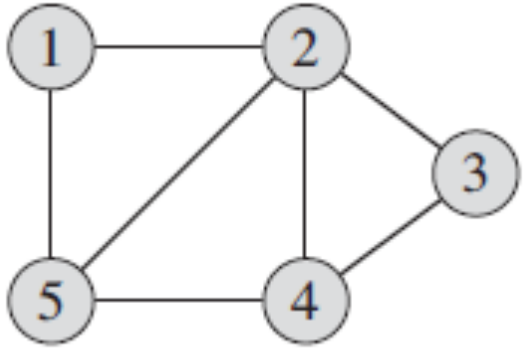
- A non-linear data structure, like a Tree.
  (Linear data structure example: Linked List)
- Used to model pairwise relations between objects.
- Mathematically, a Graph G is defined as G = {V, E}.
  V = a nonempty set of Vertices
  E = possibly nonempty set of Edges
- Vertices (also referred as Nodes) represent the “objects”
- Edges are the links/lines that “connect” vertices.



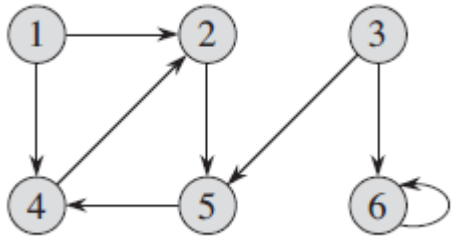


Types of Graphs

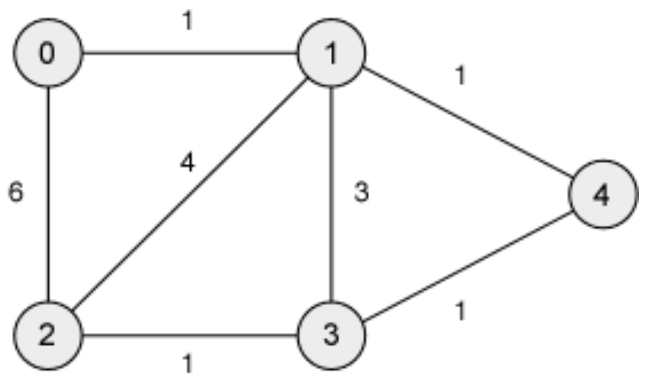
- Undirected Graphs



- Directed Graphs



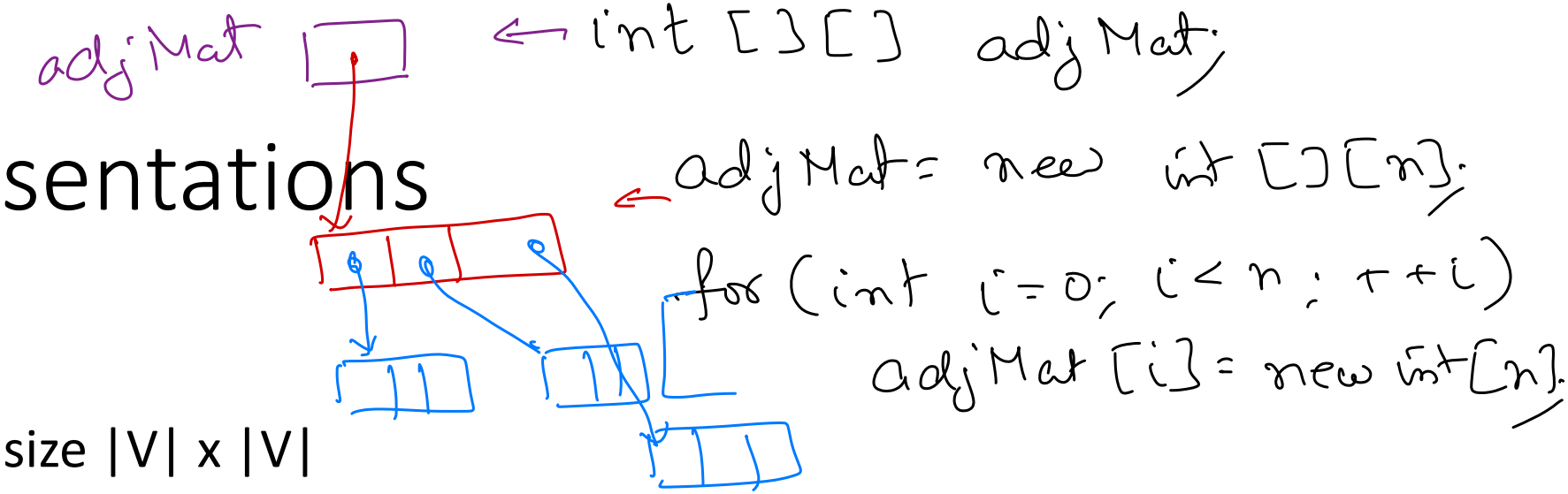
- Weighted Graphs



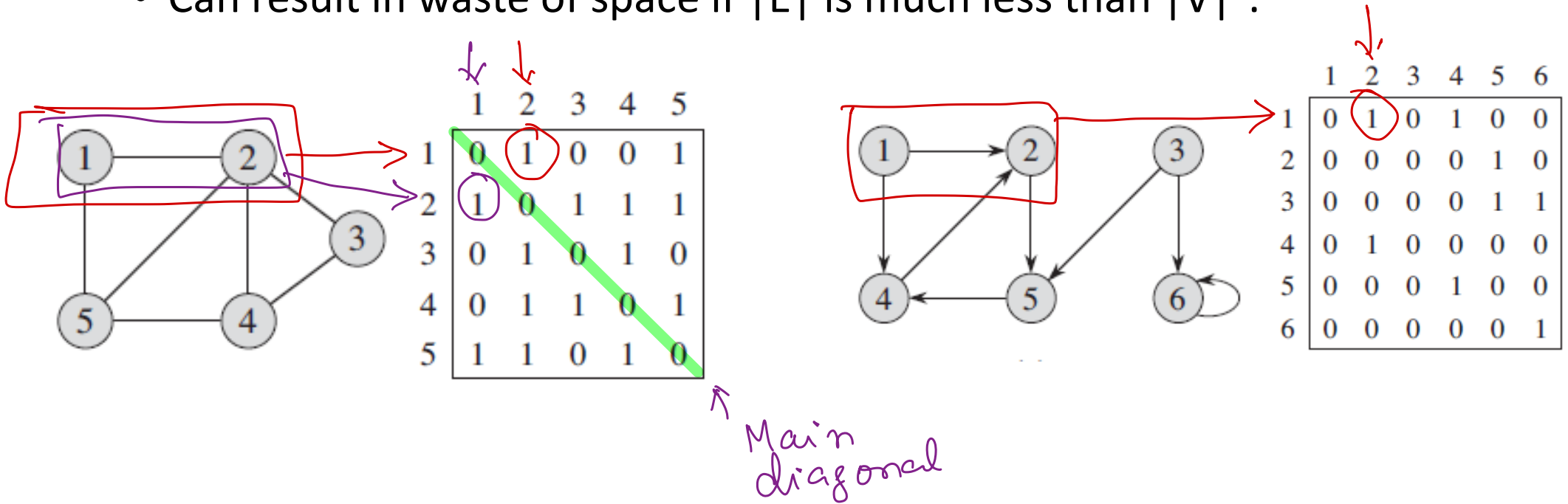
Application of Graphs

- Graphs are used to represent “flow”.
- To represent a map where roads are edges and the intersection of roads is a vertex. Example of directed weighted graph. Navigation systems can use shortest path algorithm to find shortest path between two points.
- Facebook users are considered as vertex and an edge exists between two users if they are “friends”. Example of undirected graph.
- In Operating System, used for Job Scheduling and Resource Allocation to detect deadlock.

Graph Representations



- Adjacency Matrix
  - It is a 2D array of size |V| x |V|
  - Every cell a_{ij} is 1 if there exists an edge between vertex v_i and v_j and 0, otherwise.
  - Can result in waste of space if |E| is much less than $|V|^2$.



JAVA Collection classes

List < List < EdgeInfo >> adjList = new ArrayList();

Graph Representations

class EdgeInfo {
int adjVertex;
int edgeWeight;
}

for (int i = 0; i < n; ++i)
adjList.get(i).
= new Array

- Adjacency List
  - It is a 1D array of |V| lists. One list for each vertex.
  - For each vertex *u*, the adjacency list contains all vertices *v*, such that there is an edge between vertices *u* and *v*.